Лекция 6. Глубокое обучение

Технологии и инструментарий анализа больших данных

Курс ТИАБД



Раздел

Позиция лекции в курсе



Основная идея

Предыдущая лекция показала, как текст превращается в числовое пространство с помощью **Bag of Words**, n-граммы, **TF–IDF**, плотных векторных представлений слов.

Теперь обсуждаем следующий шаг, а именно как модель может не только получать признаки, но и **обучать внутреннее представление данных**.

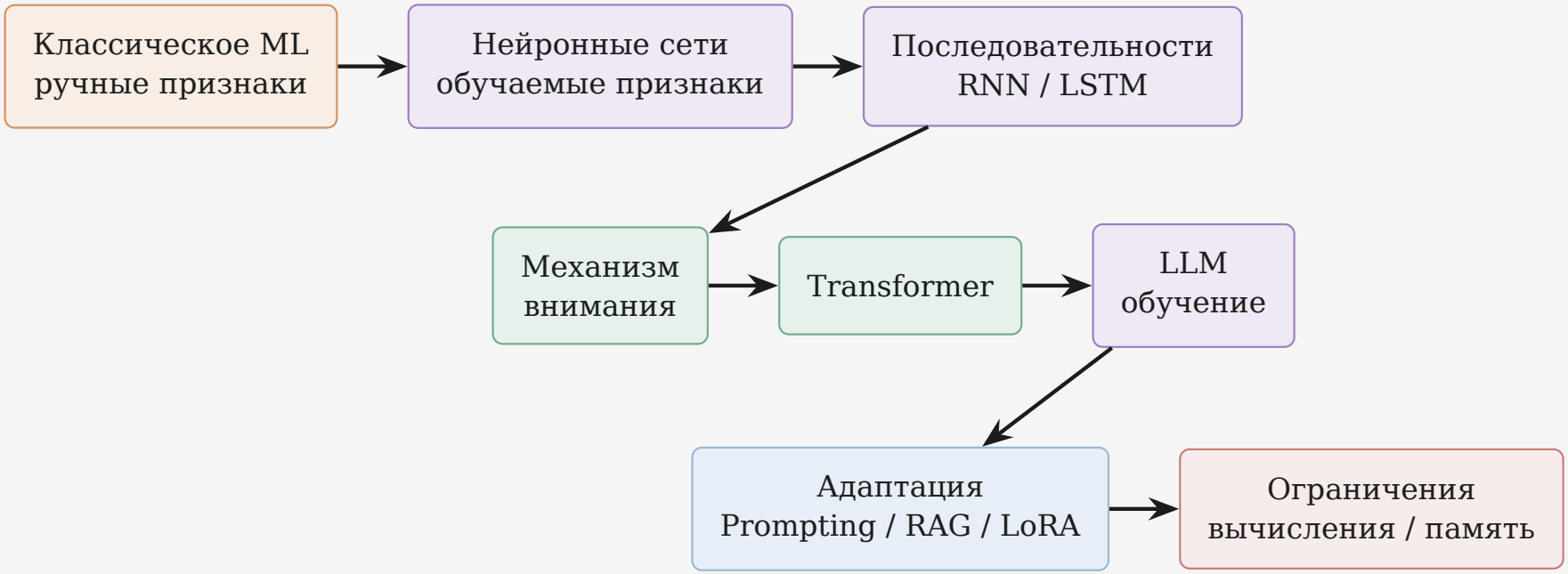
Современный NLP

Современный NLP возникает из последовательной эволюции от нейрона и многослойной сети к обработке последовательностей, механизму внимания, Transformer и большим языковым моделям.

Цель не в том, чтобы использовать LLM как «чёрный ящик», а в том, чтобы понимать архитектурные и вычислительные причины их появления.



Логика лекции



Каждый следующий блок появляется как ответ на ограничение предыдущего: слабые ручные признаки, трудность длинного контекста, дорогая последовательная обработка, стоимость обучения и внедрения.

Что должно быть понятно после лекции

1. чем глубокое обучение отличается от классического ML на уровне представлений;
2. как формально устроены нейрон, слой, loss, gradient descent и backpropagation;
3. почему текст как последовательность приводит к моделям RNN/LSTM и почему они ограничены;
4. как работает scaled dot-product attention;
5. из каких блоков состоит Transformer;
6. чем различаются encoder, decoder и encoder–decoder;
7. как связаны pretraining, fine-tuning, PEFT/LoRA, quantization, prompting и RAG;
8. почему LLM являются задачей Big Data не только по объёму текста, но и по ресурсам.



Раздел

От классического ML к Deep Learning



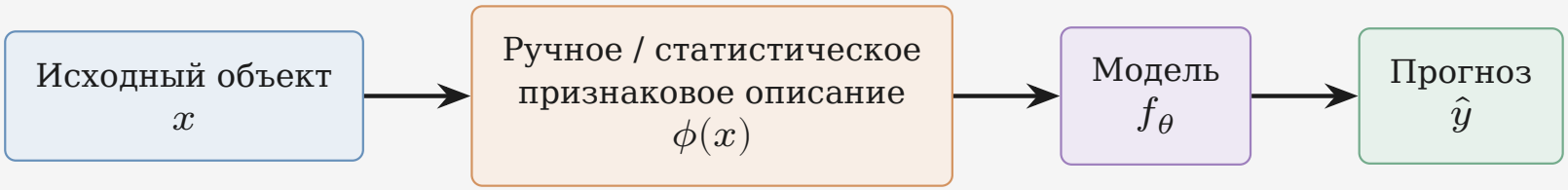
Классическое ML от признаков до модели

В традиционной постановке объект x сначала преобразуется в вектор признаков:

$$x \to \phi(x),$$

после чего обучается модель с параметрами θ:

$$x \to \phi(x) \to f_\theta(\phi(x)) \to \hat{y}.$$



Для текста примерами $\phi(x)$ были BoW, TF–IDF, n-граммы, статистики документа и заранее рассчитанные векторные представления.



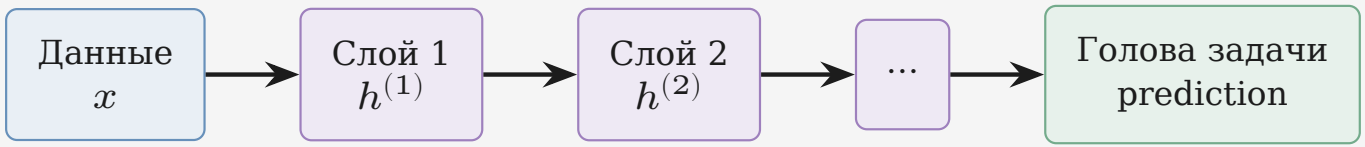
Deep Learning как представление становится обучаемым

В глубоком обучении значительная часть признакового пространства строится самой моделью:

$$x \to h_{\theta_1}(x) \to f_{\theta_2}(h) \to \hat{y}.$$

Тезис

Deep Learning не отменяет признаки. Он делает существенную часть признакового представления обучаемой и оптимизирует её совместно с целевой задачей.





Ограничения классического обучения

Классические текстовые признаки имеют ограничения:

- BoW почти не хранит порядок слов;
- n-граммы учитывают только локальный контекст;
- одно слово часто получает одно представление независимо от смысла;
- размерность словаря очень велика, а векторы разрежены;
- семантические отношения приходится задавать косвенно.

Пример

Слово «ключ» в выражениях «ключ от двери» и «ключ к решению» должно иметь разные контекстные представления. Модель должна учитывать окружение слова, а не только его идентификатор.



Deep Learning как задача Big Data

Современная нейросетевая модель требует одновременно:

Данные

- большие корпуса текстов;
- очистка и дедупликация;
- токенизация;
- контроль качества и смещений.

Вычисления

- GPU/TPU или кластеры;
- память для параметров и активаций;
- сетевой обмен;
- хранение состояний.

качество модели $\approx f(\text{данные, архитектура, вычисления, обучение})$.



Раздел

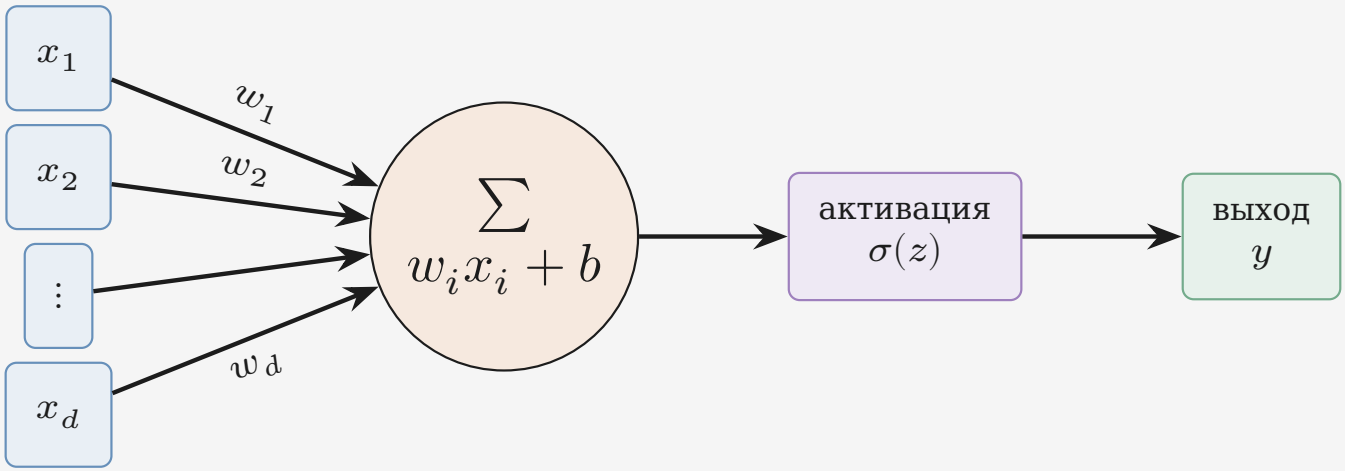
Нейрон, слой и обучение



Искусственный нейрон

Пусть объект описан вектором $x \in \mathbb{R}^d$. Нейрон имеет веса $w \in \mathbb{R}^d$ и смещение b:

$$z = w^T x + b = \sum_{i=1}^{d} w_i x_i + b, \qquad y = \sigma(z).$$



Зачем нужна нелинейная активация

Если два слоя линейны,

$$h = W_1 x + b_1, \qquad y = W_2 h + b_2,$$

то после подстановки:

$$y = W_2(W_1 x + b_1) + b_2 = (W_2 W_1)x + (W_2 b_1 + b_2) = W' x + b'.$$

Следствие

Композиция линейных слоёв остаётся линейным преобразованием. Без нелинейности глубина сети не даёт принципиально новой выразительности.

Примеры активаций:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \tanh(z), \qquad ReLU(z) = \max(0, z).$$

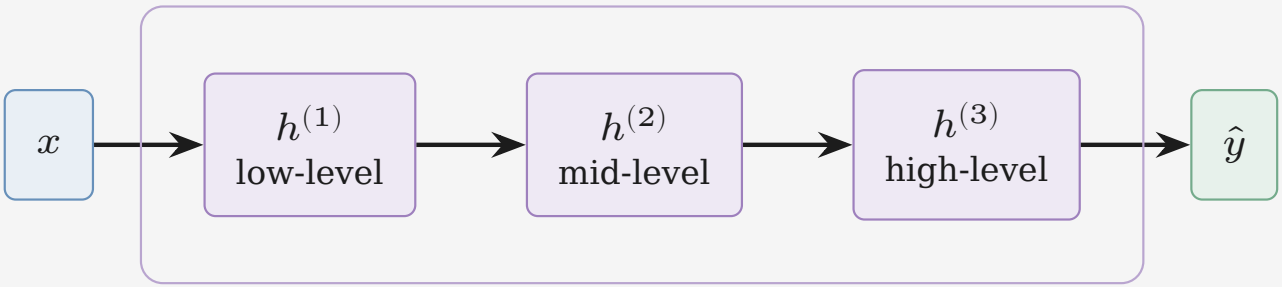


Многослойная сеть

Слой нейронной сети можно записать как:

$$h^{(l)} = \sigma\left(W^{(l)} h^{(l-1)} + b^{(l)}\right).$$

Здесь $h^{(0)} = x$, а $h^{(l)}$ — внутреннее представление на l-м слое.



Функция потерь

Обучение требует численно измерить ошибку модели. Для регрессии часто используют MSE:

$$L(\theta) = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2.$$

Для классификации обычно используется кросс-энтропия:

$$L(\theta) = -\sum_{c=1}^{C} y_c \log p_c.$$

Интерпретация

Функция потерь превращает качество решения в скалярную величину, которую можно минимизировать по параметрам модели.

$$\theta^* = \arg\min_{\theta} L(\theta).$$

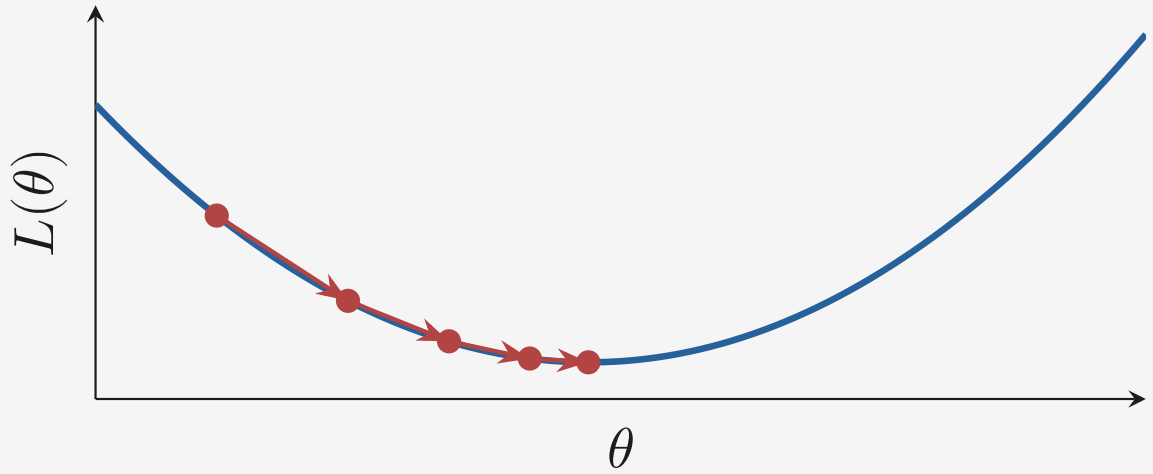


Градиентный спуск

Обновление параметров выполняется в направлении антиградиента:

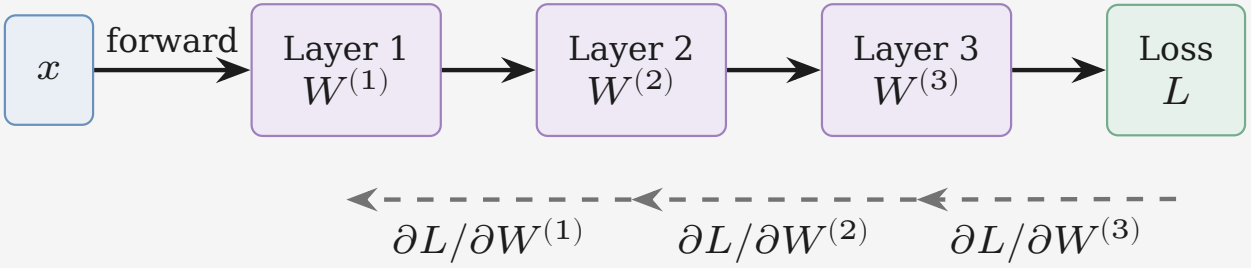
$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t),$$

где η — скорость обучения.



Обратное распространение ошибок

Backpropagation вычисляет производные функции потерь по параметрам всех слоёв, используя правило цепочки.



Смысл

Прямое распространение строит прогноз и считает ошибку, *обратное распространение* вычисляет, как каждый параметр повлиял на эту ошибку.



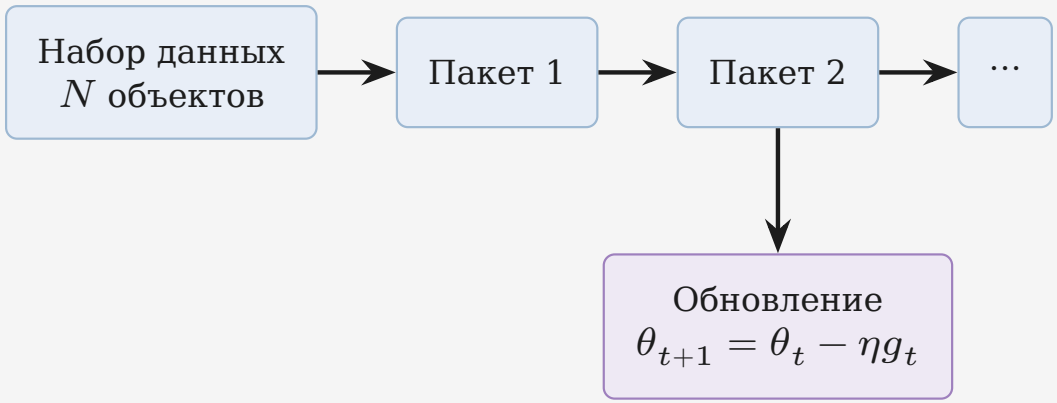
Мини-пакетное обучение

На больших данных параметры обычно обновляют не по всему датасету сразу, а по мини-пакетам:

$$B_t = \{(x_i, y_i)\}_{i=1}^{m}.$$

Оценка градиента становится стохастической:

$$\nabla L(\theta) \approx \frac{1}{m} \sum_{(x_i, y_i) \in B_t} \nabla \ell(f_\theta(x_i), y_i).$$





Раздел

Последовательности и проблема контекста



Почему текст сложнее таблицы

Текст является последовательностью:

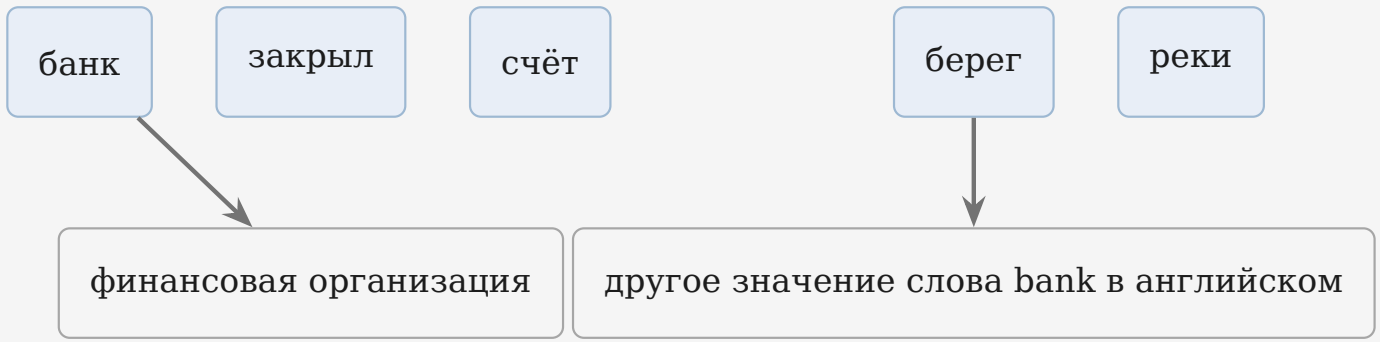
$$d = (w_1, w_2, \dots, w_T).$$

Для языковой модели типичная задача:

$$P(w_t \mid w_1, \dots, w_{t-1}).$$

Проблема

Модель должна хранить порядок, дальние зависимости и контекстное значение слов. Простое множество токенов для этого недостаточно.

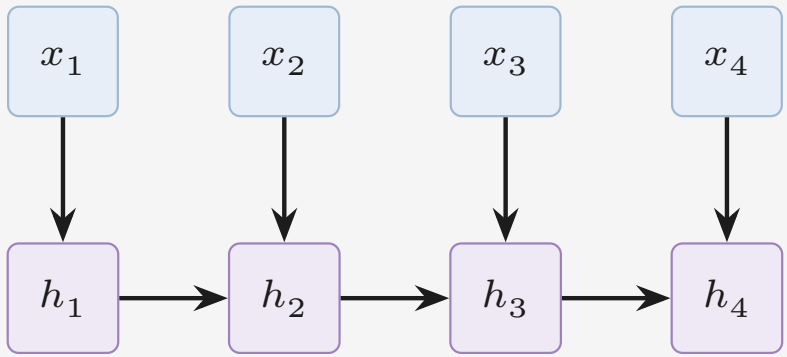




RNN обработка слева направо

Рекуррентная сеть обновляет скрытое состояние при чтении каждого токена:

$$h_t = f(W_x x_t + W_h h_{t-1} + b).$$

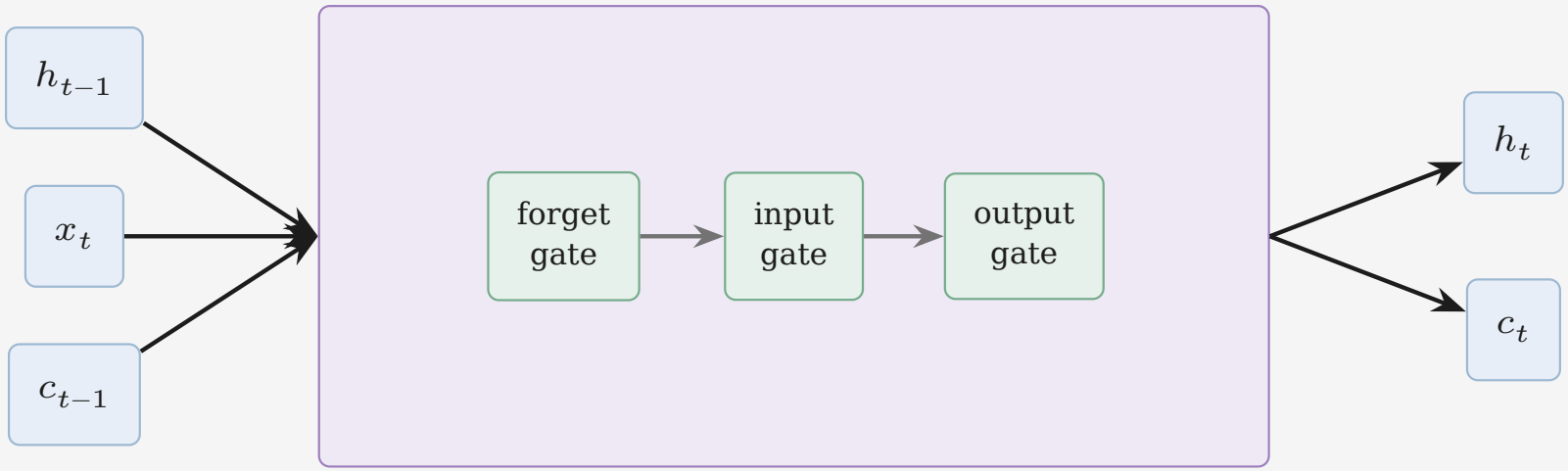


RNN естественно кодирует порядок, но плохо масштабируется по длине последовательности и плохо параллелится.



LSTM память и забывание

LSTM добавляет управляемую память. Она определяет какую информацию сохранить, какую забыть, какую выдать наружу.



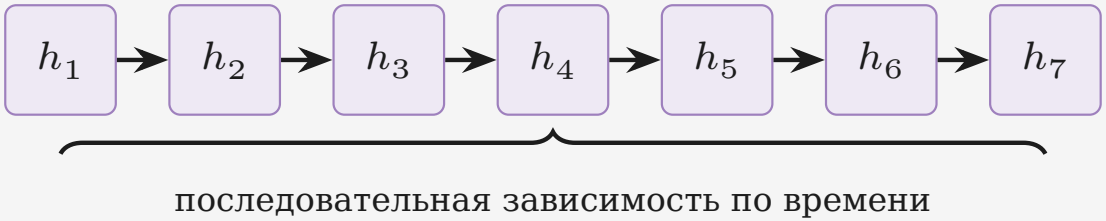
Идея

LSTM лучше работает с долгими зависимостями, но всё равно обрабатывает последовательность рекуррентно и поэтому имеет ограниченную параллельность.

Ограничение рекуррентной обработки

Для последовательности длины T модель RNN вынуждена пройти состояния последовательно:

$$h_1 \to h_2 \to \ldots \to h_T.$$



Инженерный вывод

Если модель должна обучаться на огромных корпусах, архитектура с сильной последовательной зависимостью становится узким местом.



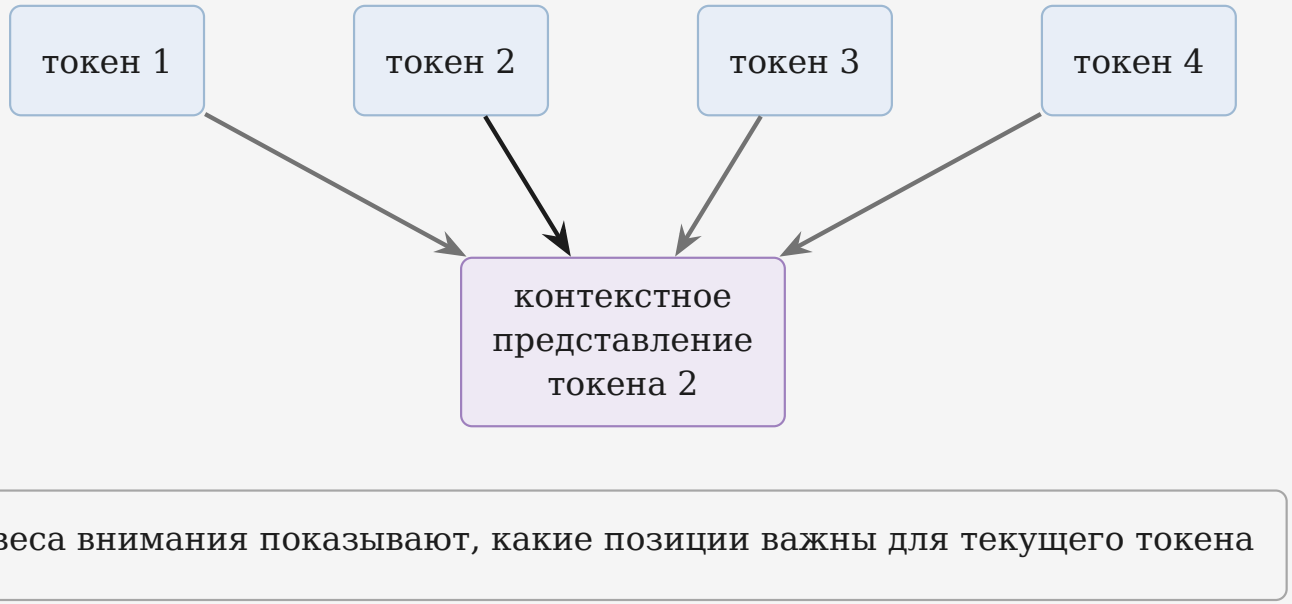
Раздел

Механизм внимания



Основная идея внимания

Механизм внимания позволяет каждому токену напрямую смотреть на другие токены и формировать контекстное представление.





Запрос, Ключ, Значение

Для каждого токена строятся три вектора:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$

Query

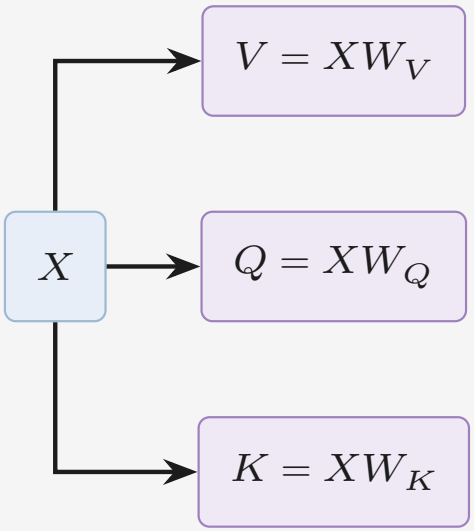
Что текущая позиция ищет в контексте?

Key

С чем другие позиции могут быть сопоставлены?

Value

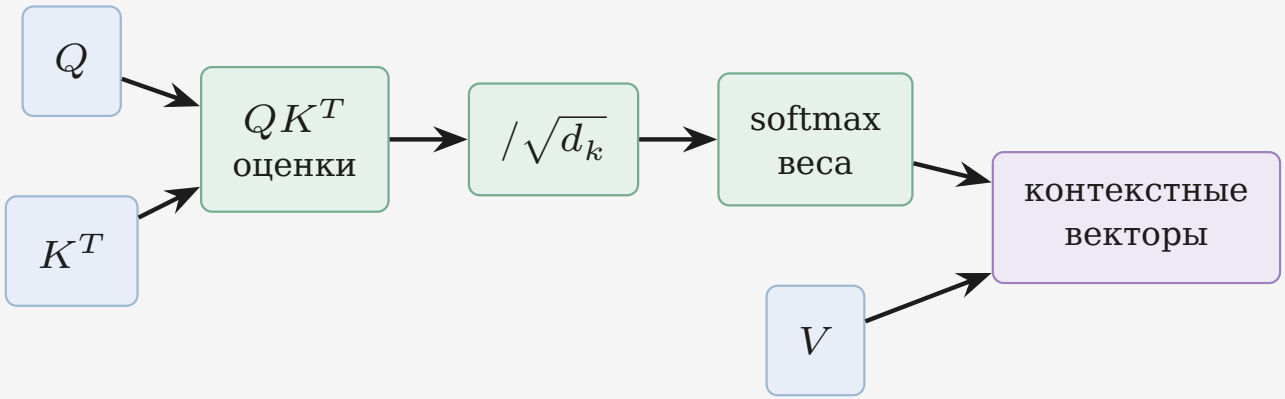
Какая информация будет передана при высокой релевантности?



Масштабированное скалярное внимание

Центральная формула:

$$\text{Attention}(Q, K, V) = \text{SM}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad SM_j = \frac{exp(X_j)}{\sum_k exp(X_k)}$$

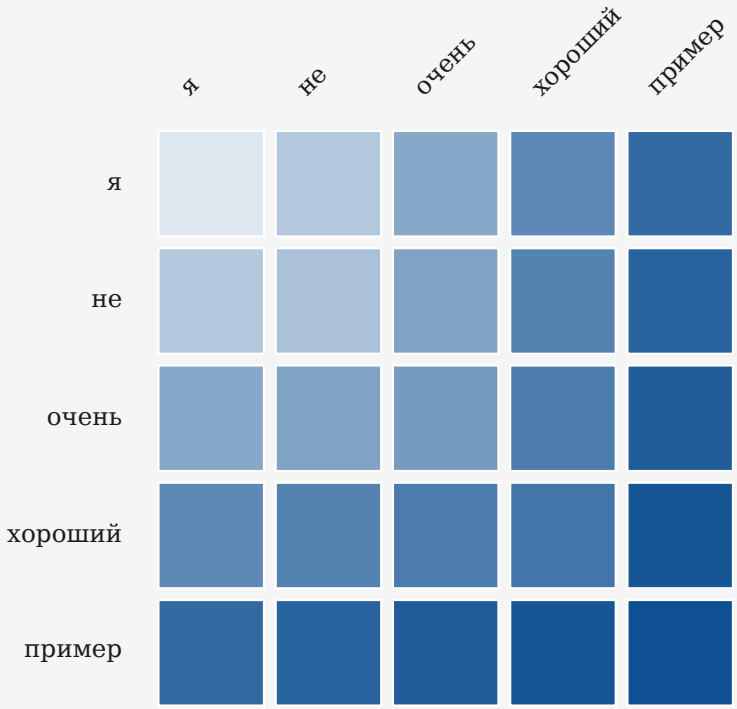


Деление на $\sqrt{d_k}$ стабилизирует масштаб скалярных произведений перед softmax.

Матрица внимания

Матрица $A = \text{softmax}(QK^T/\sqrt{d_k})$ имеет размер $T \times T$:

A_{ij} = насколько токен i обращает внимание на токен j.



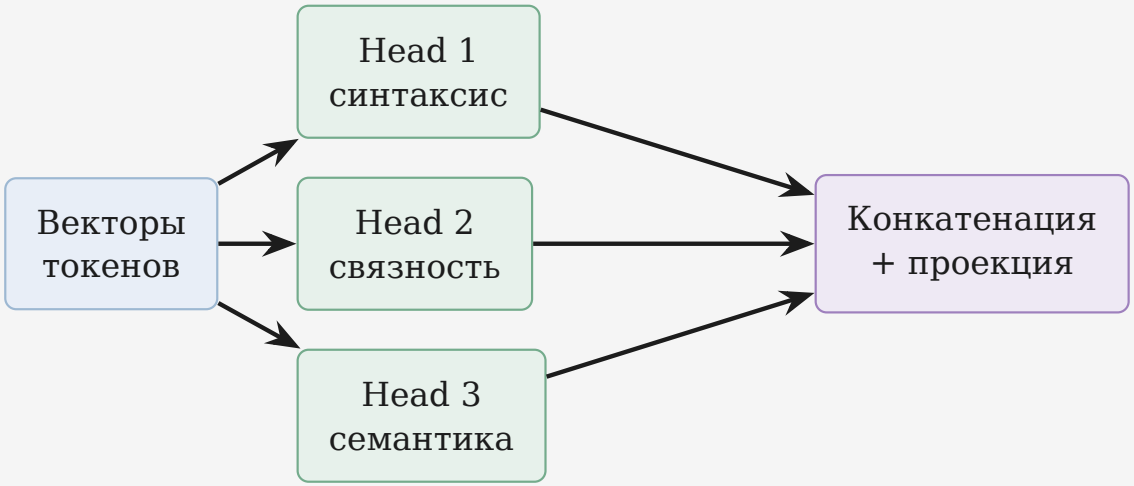
условная heatmap весов attention

Многоголовое внимание

Одна голова внимания может выделять один тип отношений. Несколько голов позволяют параллельно учить разные виды зависимостей:

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V),$$

$$MultiHead = Concat(head_1, \dots, head_h)W^O.$$



Самовнимание и вычислительная стоимость

Self-attention сопоставляет все позиции со всеми:

$$QK^T \in \mathbb{R}^{T \times T}.$$

По длине последовательности стоимость базового внимания имеет порядок:

$$O(T^2 d).$$

Преимущество

- прямые связи между токенами;
- хорошая параллельность;
- сильные контекстные представления.

Цена

- квадратичная матрица внимания;
- большой расход памяти;
- сложность длинного контекста.



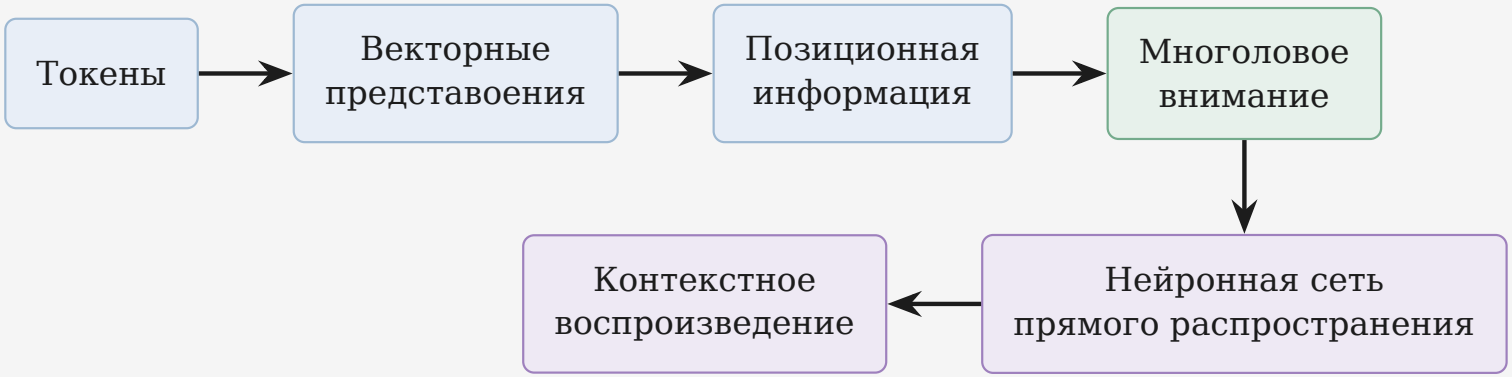
Раздел

Transformer



Архитектурный переход к Transformer

Transformer отказывается от рекуррентного прохода как основного механизма и строит представления через attention и полносвязные преобразования.



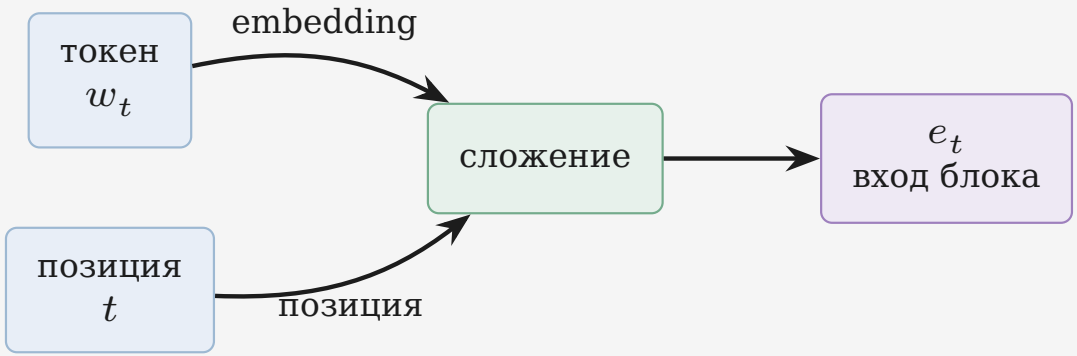
Мотивация

Сделать обработку последовательностей более параллельной и дать каждому токену доступ к контексту всей последовательности.

Зачем нужны позиционные представления

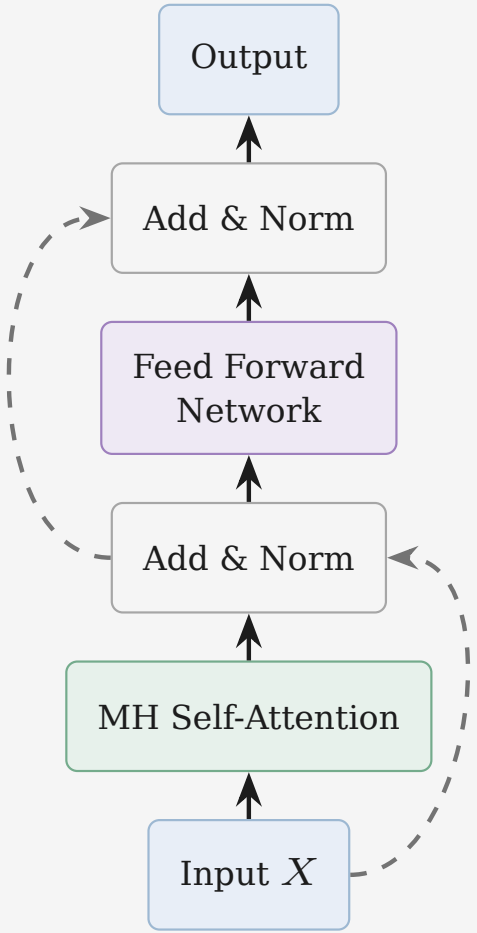
Self-attention сам по себе не знает порядка токенов, поэтому множество токенов можно перемешать. Операция попарного сравнения не задаёт позицию.

$$e_t = token_embedding(w_t) + position_embedding(t).$$





Один блок Transformer



Пунктир показывает разностные соединения. Они облегчают обучение глубоких сетей и передачу градиента.

Feed Forward Network внутри Transformer

После attention каждый токен отдельно проходит через одинаковую MLP:

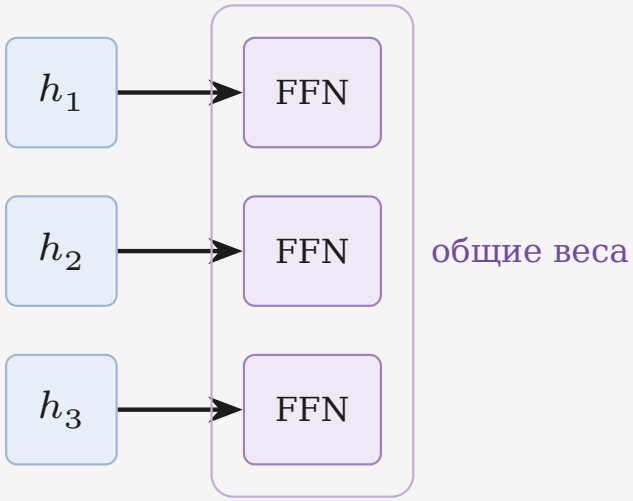
$$FFN(x) = W_2\,\sigma(W_1 x + b_1) + b_2.$$

Attention

Смешивает информацию между позициями последовательности.

FFN

Нелинейно преобразует представление каждой позиции.



Encoder, decoder, encoder–decoder

Encoder

Строит представление входного текста.

Задачи: классификация, NER, векторные представления.

Decoder

Генерирует последовательность авторегрессионно:

$$P(w_t \mid w_{<t}).$$

Encoder–Decoder

Преобразует входную последовательность в выходную. Примеры: перевод, суммаризация.

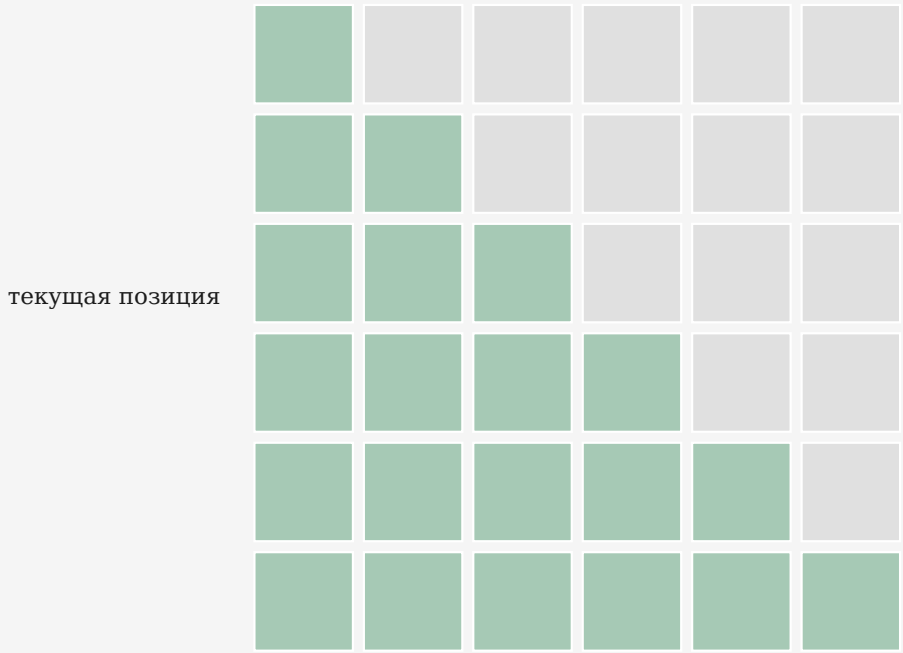




Авторегрессионная маска в decoder

При генерации токен t не должен видеть будущие токены:

$$P(w_t \mid w_1, \dots, w_{t-1}).$$



зелёная область доступна, серая область замаскирована

Именно decoder с маской лежит в основе большинства генеративных языковых моделей.

Раздел

Pretraining и большие языковые модели



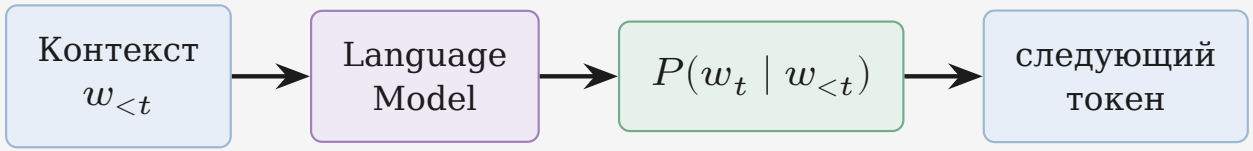
Что такое языковая модель

Языковая модель оценивает вероятность последовательности или следующего токена:

$$P(w_1, \ldots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_{<t}).$$

В генеративной модели

Следующий токен выбирается на основе распределения вероятностей, полученного из текущего контекста.

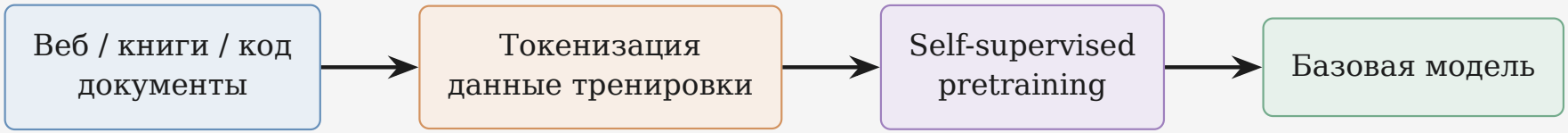




Pretraining

Pretraining — обучение общей модели на большом корпусе до решения конкретной прикладной задачи.

Большой корпус → обобщенная модель.



Self-supervised означает, что обучающий сигнал извлекается из самих данных.

Например, предсказать следующий или скрытый токен.



Почему pretraining работает

В процессе решения простой формальной задачи модель вынуждена усваивать статистические зависимости языка:

- частотность и совместную встречаемость слов;
- синтаксические шаблоны;
- типичные отношения между сущностями;
- фактические фрагменты знаний из корпуса;
- стили и форматы текстов;
- элементы рассуждения, если они представлены в данных.

Важно

Модель не хранит базу данных в явном виде. Она параметризует распределение текста, обученное на корпусе.



Масштаб параметров и данных

Пусть модель имеет P параметров, корпус содержит N токенов, а размер скрытого представления равен d.

Параметры

Веса attention, FFN, embeddings и нормализаций.

$$P \gg 10^6$$

Токены

Обучающие примеры языкового распределения.

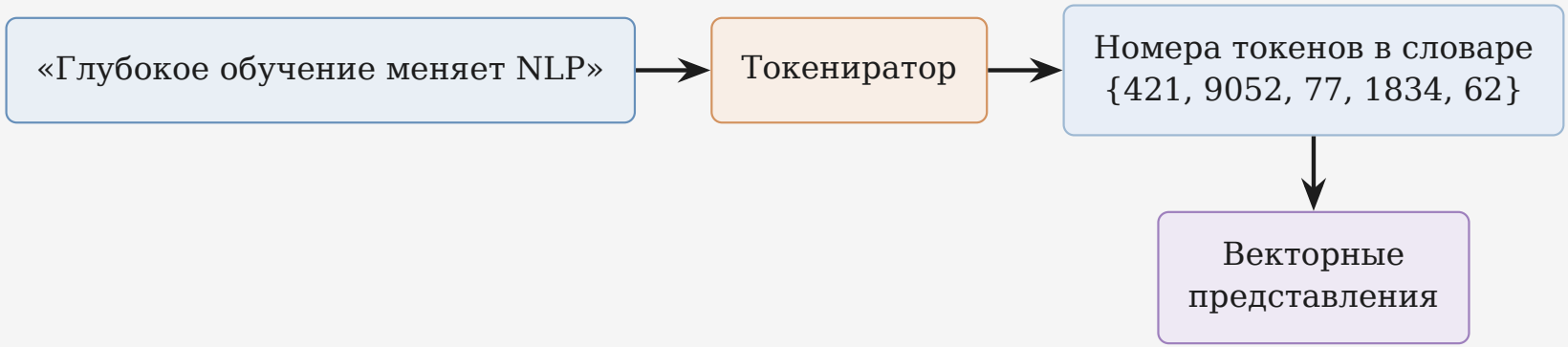
$$N \gg 10^9$$

стоимость обучения $\sim$ количество операций на token $\times\, N$.



Токенизация текста

Модель работает не с символами и не со словами в обычном смысле, а с токенами.



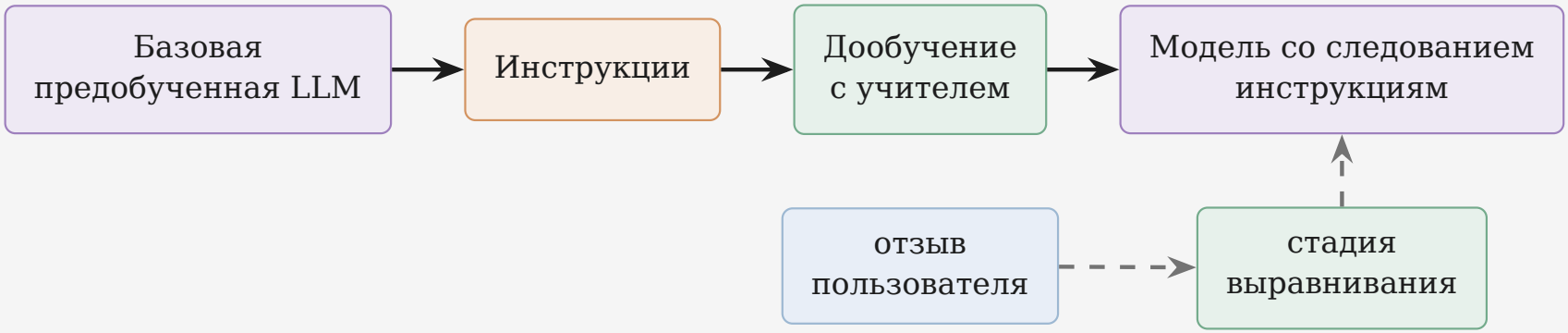
Следствие

Длина контекста, стоимость внимания и качество работы с редкими словами зависят не только от архитектуры, но и от токенизатора.



От базовой модели к ассистенту

Базовая модель хорошо моделирует продолжение текста, но это не всегда означает удобное следование инструкциям.



Раздел

Адаптация моделей



Три способа использовать LLM в задаче

Prompting

Параметры модели не меняются. Меняется только входной запрос и контекст.

Fine-tuning

Параметры модели дополнительно обучаются на доменном датасете.

RAG

Модель получает релевантные документы из внешнего хранилища.

Разграничение

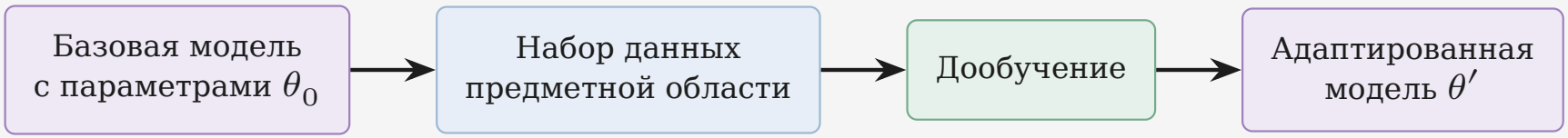
Prompting не изменяет модель. Fine-tuning изменяет параметры. RAG изменяет контекст за счёт получения релевантных данных из БД, но обычно не изменяет веса модели.



Дообучение

Fine-tuning адаптирует предобученную модель к задаче или домену:

$$\theta_0 \to \theta_0 + \Delta\theta.$$



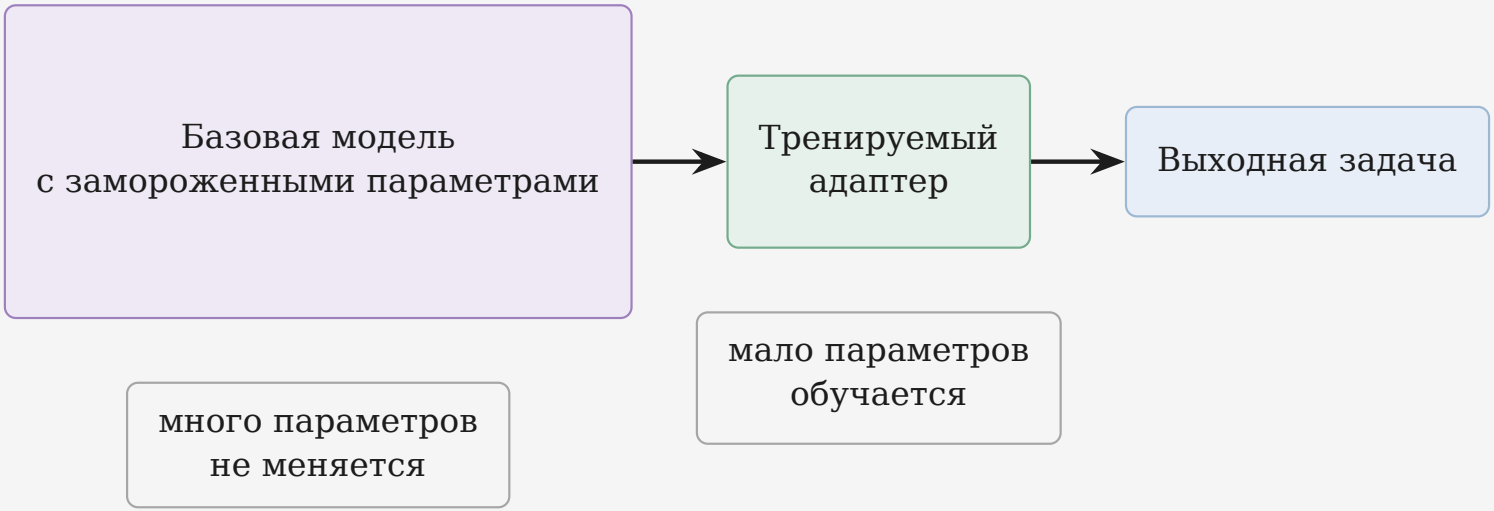
Цена

Полное дообучение требует памяти под веса, градиенты, optimizer states и активации. Для крупных моделей это часто недоступно на обычных студенческих устройствах.



PEFT. Параметро-эффективная адаптация

PEFT-подходы обучают не все параметры модели, а небольшое число дополнительных или выбранных параметров.



Смысл

Снижаются требования к памяти и времени обучения, а базовая модель остаётся общей.

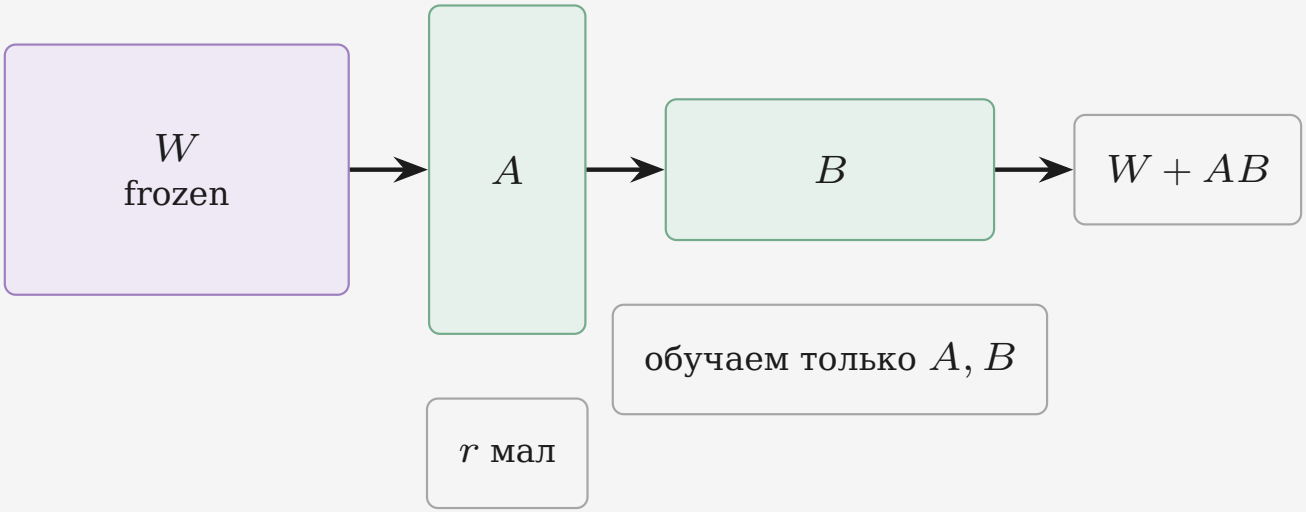
LoRA. Низкоранговая аппроксимация

Вместо полного изменения матрицы весов:

$$W' = W + \Delta W$$

LoRA задаёт поправку как произведение низкоранговых матриц:

$$\Delta W = AB, \qquad rank(AB) \ll \min(m, n).$$



Квантизация

Quantization уменьшает точность хранения чисел, чтобы снизить объём памяти и иногда ускорить вычисление предсказания.

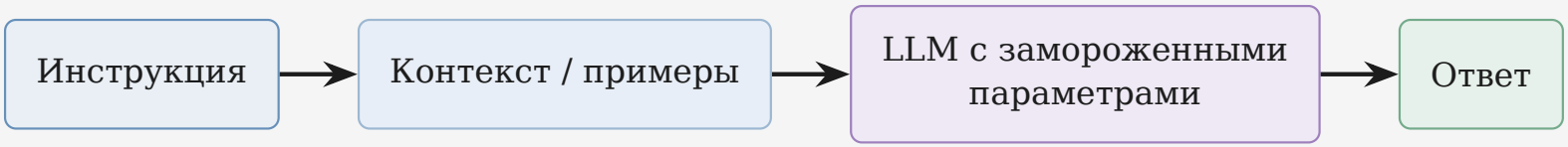


память $\downarrow$, качество, может $\downarrow$, задержка, часто $\downarrow$.



Промптинг

Prompting — управление поведением модели через входной текст без изменения параметров.



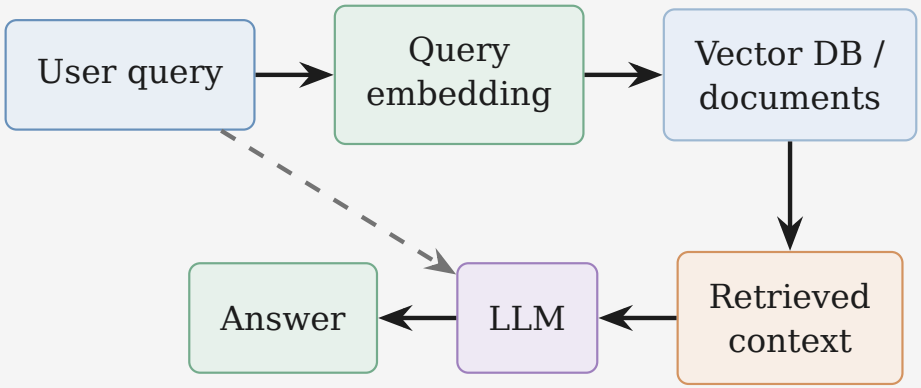
Ограничение

Промпт может направлять модель, но не добавляет в её параметры новые проверенные знания и не гарантирует корректность фактов.



RAG: Retrieval-Augmented Generation

RAG соединяет поиск по внешнему корпусу с генерацией ответа.



$$Query \to Retrieval \to Context \to LLM.$$

Когда выбирать prompting, RAG, LoRA или fine-tuning

Подход	Что меняется	Когда полезен	Цена
Prompting	входной текст	быстрый сценарий, формат ответа	низкая
RAG	контекст из базы знаний	нужны актуальные документы и ссылки на источники	средняя
LoRA / PEFT	малая добавка параметров	доменный стиль, специальная задача	средняя
Full fine-tuning	много параметров модели	глубокая смена поведения модели	высокая

Практическое правило

В условиях ограниченных ресурсов сначала рассматривают prompting и RAG, затем PEFT/LoRA, и только после этого полное fine-tuning.



Раздел

Вычислительные ограничения и внедрение



Почему мы не обучаем LLM с нуля в учебной практике

Написать цикл обучения сравнительно просто. Трудность находится в масштабе:

- нужны огромные корпуса и сложная подготовка данных;
- требуются ускорители и распределённое обучение;
- занимаемая память включает веса, градиенты, стадии оптимизатора, активации;
- длительное обучение требует контроля сбоев и точек сохранения;
- оценка качества и безопасности модели становится отдельной задачей.



Грубая оценка памяти модели

Если модель имеет P параметров, то только хранение весов требует приблизительно:

$$M_{weights} = P \cdot b,$$

где b — число байт на параметр.

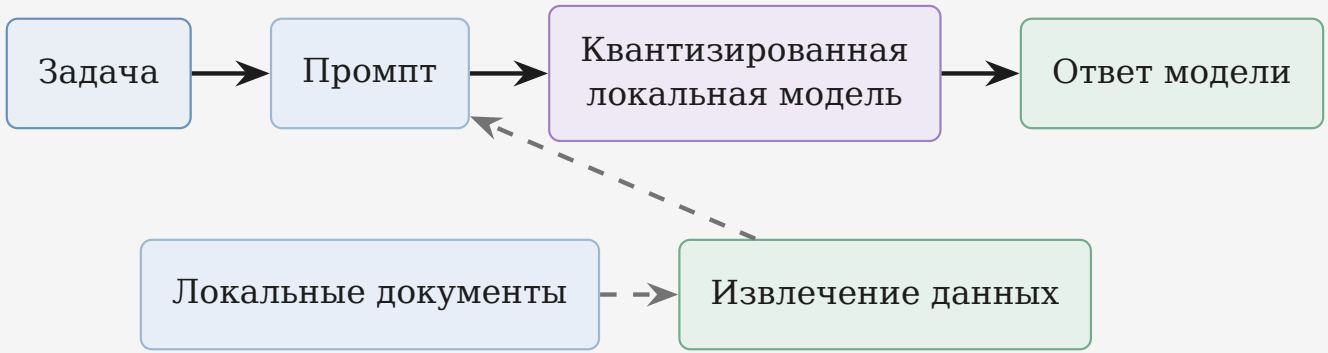
Формат	байт на параметр	пример для $P = 7 \cdot 10^9$
FP32	4	≈ 28 GB
FP16/BF16	2	≈ 14 GB
INT8	1	≈ 7 GB
INT4	0.5	≈ 3.5 GB

При обучении памяти требуется существенно больше, чем для одних весов: добавляются градиенты, состояния оптимизатора и промежуточные активации.



Inference пайплайн для локального эксперимента

Даже без обучения можно собрать пайплайн для использования предобученной модели.

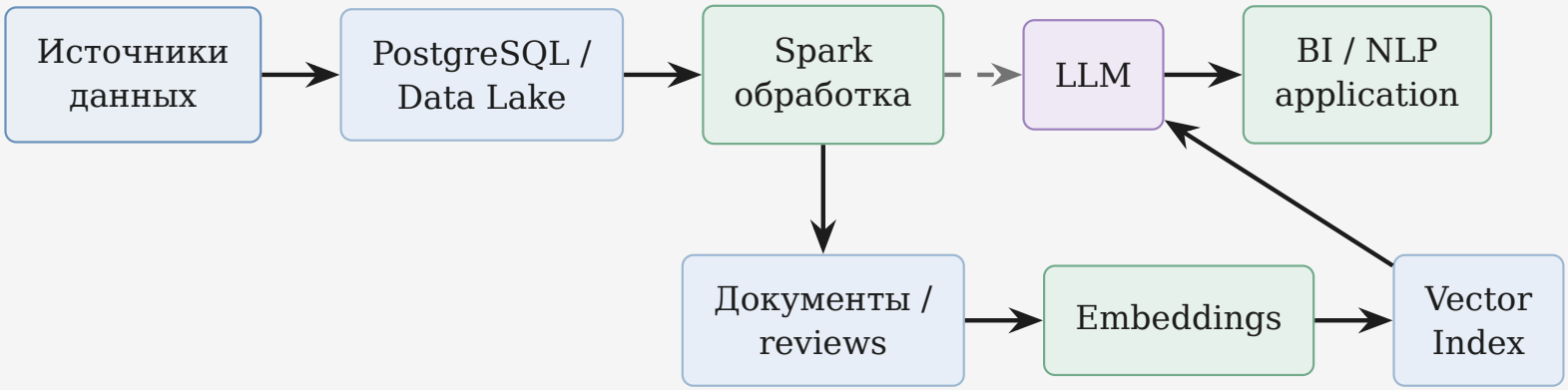


Связь с курсом

Такой пайплайн использует хранение документов, векторные представления, извлечение релевантных данных, модель и оценку результата — то есть соединяет инженерию данных и современный ИИ.



LLM в архитектуре аналитической системы



LLM становится одним из компонентов аналитической платформы.



Риски и ограничения современных языковых моделей

Технические

- задержка ответа и стоимость инференса;
- ограничение длины контекста;
- расход памяти;
- нестабильность ответов.

Методологические

- галлюцинации;
- смещения данных;
- отсутствие гарантии истинности;
- сложность оценки качества.

Для аналитика

Ответ модели должен рассматриваться как результат вероятностной системы, а не как автоматически достоверный факт.



Раздел

Итоги лекции



Сравнение подходов к тексту

Подход	Представление	Что хорошо делает	Ограничение
BoW / TF–IDF	sparse vector	быстрый baseline, поиск слов	слабый контекст
Word2Vec	dense word vector	семантическая близость слов	одно значение на слово
RNN / LSTM	hidden state	порядок и последовательность	плохая параллельность
Transformer	contextual embeddings	дальний контекст и масштаб	дорогой attention
LLM	pretrained Transformer	генерация и перенос знаний	ресурсы и контроль качества



Главные определения лекции

Deep Learning обучение многоуровневых представлений данных нейронной сетью.

Attention механизм взвешенного обращения одной позиции последовательности к другим позициям.

Transformer архитектура, основанная на self-attention, FFN, residual connections и positional information.

Pretraining обучение общей модели на большом корпусе до прикладной адаптации.

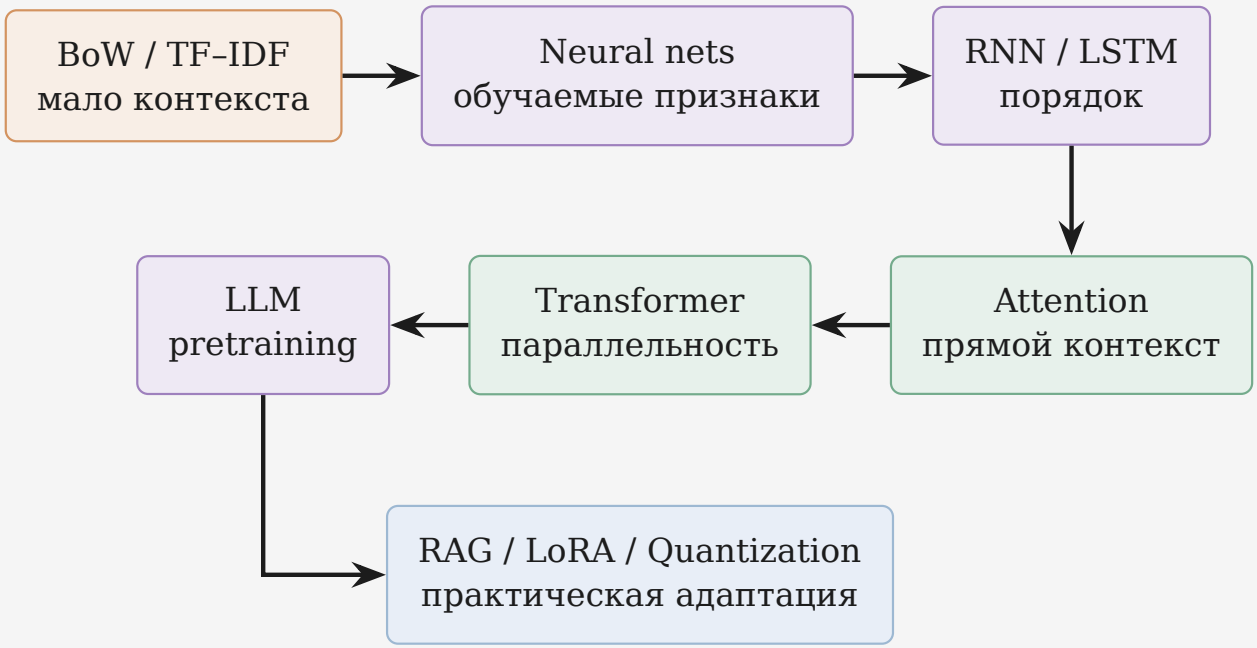
Fine-tuning изменение параметров модели под домен или задачу.

LoRA PEFT-метод, обучающий низкоранговую поправку к весовым матрицам.

RAG генерация с внешним извлечением релевантного контекста.



Развитие методов обработки текстовой информации



Финальный тезис

Современные языковые модели — это не отдельный «чат-бот», а результат совместной эволюции представлений, архитектур, данных и распределённых вычислений.

Контрольные вопросы

1. Почему композиция линейных слоёв остаётся линейной?
2. Что именно обучается в deep learning по сравнению с классическим ML?
3. Почему RNN трудно масштабировать на очень длинные последовательности?
4. Как интерпретировать матрицу QK^T в attention?
5. Зачем attention делится на $\sqrt{d_k}$?
6. Чем encoder отличается от decoder?
7. Почему pretraining называют self-supervised?
8. В чём различие между prompting, RAG и fine-tuning?
9. Почему LoRA снижает стоимость адаптации?
10. Почему обучение LLM с нуля не является реалистичной задачей для обычной учебной практики?

